

开源软件基础

Python 动态分析与静态分析

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

December 28, 2025



Outline

- 1 inspect
- 2 sys.settrace
- 3 sys.monitoring
- 4 tokenize
- 5 libcst

Python 的作用域

Built-in (Python)

Names preassigned in the built-in names module: open, range, SyntaxError....

Global (module)

Names assigned at the top-level of module file, or declared global in a def within the file.

Enclosing function locals

Names in the local scope of any and all enclosing functions (def or lambda), from inner to outer.

Local (function)

Names assigned in any way within a function (def or lambda), and not declared global in that function.

inspect

```
>>> import inspect
>>> def foo():
...     x = 10
...     return inspect.currentframe()
>>> f = foo()
>>> dir(foo)
['__class__',
...
'f_back',
'f_builtins',
'f_code',
'f_globals',
'f_lineno',
'f_locals',
'f_trace']
```

inspect

Table 1: frame 成员

成员	描述
f_back	next outer frame object (this frame's caller)
f_builtins	builtins namespace seen by this frame
f_code	code object being executed in this frame
f_globals	global namespace seen by this frame
f_lasti	index of last attempted instruction in bytecode
f_lineno	current line number in Python source code
f_locals	local namespace seen by this frame
f_trace	tracing function for this frame, or None

inspect

```
x = 10
def outer():
    x = 20
    def inner():
        x = 30
        print(x)
        return inspect.currentframe()
    return inner
inn = outer()
frame = inn()
print(frame.f_locals['x'])
print(frame.f_globals['x'])
print(inspect.getclosurevars(inn))
```

inspect

```
x = 10
def outer():
    x = 20
    def inner():
        # x = 30
        print(x)
        return inspect.currentframe()
    return inner
inn = outer()
frame = inn()
print(frame.f_locals['x'])
print(frame.f_globals['x'])
print(inspect.getclosurevars(inn))
```

Outline

- 1 inspect
- 2 sys.settrace**
- 3 sys.monitoring
- 4 tokenize
- 5 libcst

settrace

```
from sys import settrace

def tracer(frame, event, arg = None):
    code = frame.f_code
    func_name = code.co_name
    line_no = frame.f_lineno
    print(f"事件: {event}, 函数: {func_name}(), 行号: {line_no}")
    return tracer

def foo():
    print("in foo")

def bar():
    foo()

old_tracer = settrace(tracer)
bar()
settrace(old_tracer)
```

Outline

- 1 inspect
- 2 sys.settrace
- 3 sys.monitoring**
- 4 tokenize
- 5 libcst

背景：为什么需要 sys.monitoring ?

- sys.settrace 存在的问题：
 - 事件粒度粗（行级）
 - 全局单一回调，难以组合多个工具
 - 性能开销大
- Python 3.12 引入 sys.monitoring
- 目标：
 - 更细粒度事件
 - 多工具并存（debugger / profiler / tracer）
 - 更低运行时开销

sys.monitoring 核心机制

- 事件驱动模型 (Event-based)
- 为不同工具分配 **tool_id**
- 每个工具订阅自己关心的事件

典型事件类型：

- PY_START / PY_RETURN
- LINE
- CALL / C_CALL
- INSTRUCTION

Tracer 示例：函数调用追踪

```

from sys import monitoring
indent = 0

def on_start(code, offset):
    global indent
    indent += 1
    print(f"{indent * '>'} entering function: {code.co_name}")

def on_return(code, offset, retval):
    global indent
    print(f"{indent * '<'} leaving function: {code.co_name}, with return value: {retval}")
    indent -= 1

debugger = monitoring.use_tool_id(monitoring.DEBUGGER_ID, "debugger")
monitoring.register_callback(monitoring.DEBUGGER_ID, monitoring.events.PY_START, on_start)
monitoring.register_callback(monitoring.DEBUGGER_ID, monitoring.events.PY_RETURN, on_return)

def fib(bar):
    if bar <= 2:
        return 1
    else:
        return fib(bar - 1) + fib(bar - 2)

monitoring.set_events(monitoring.DEBUGGER_ID, monitoring.events.PY_START |
                    monitoring.events.PY_RETURN)

fib(10)

monitoring.set_events(monitoring.DEBUGGER_ID, 0)
monitoring.free_tool_id(monitoring.DEBUGGER_ID)

```

Debugger / Profiler 场景

Debugger:

- 监听 LINE / INSTRUCTION 事件
- 支持断点、单步执行

Profiler:

- 监听 CALL / RETURN
- 构建调用栈、统计时间

Tracer 示例代码:

```
# 调试器和性能分析器可以同时启用  
# 而不会覆盖彼此的回调逻辑
```

sys.monitoring vs sys.settrace

维度	sys.settrace	sys.monitoring
事件粒度	行级	行 / 指令 / 调用
多工具支持	否	是 (tool_id)
性能	较高开销	更低
设计年代	Python 早期	Python 3.12+
适用场景	简单调试	可观测性 / 运行时分析

sys.monitoring 支持更高性能的 tracing

可同时用于调试和性能分析

总结

- sys.monitoring 是 Python 运行时可观测性的基础设施
- 更适合：
 - 高性能 tracing
 - 组合式调试与分析工具
 - 系统级行为建模
- 可作为：
 - 动态分析工具
 - 运行时剖析框架
 - 可观测性驱动优化的底层支撑

Outline

- 1 inspect
- 2 sys.settrace
- 3 sys.monitoring
- 4 tokenize**
- 5 libcst

词法分析

```
import tokenize
import io
code='''\
def foo(bar):
    return bar + 1
'''

for token in tokenize.tokenize(io.BytesIO(code.encode('utf-8')).readline):
    print(token)

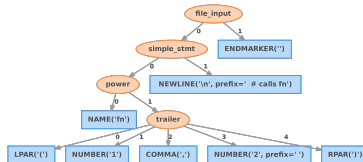
# or with code in file
with open('a.py', 'rb') as f:
    for token in tokenize.tokenize(f.readline):
        print(token)
```

```
TokenInfo(type=63 (ENCODING), string='utf-8', start=(0, 0), end=(0, 0), line='')
TokenInfo(type=1 (NAME), string='def', start=(1, 0), end=(1, 3), line='def foo(bar):\n')
TokenInfo(type=1 (NAME), string='foo', start=(1, 4), end=(1, 7), line='def foo(bar):\n')
TokenInfo(type=54 (OP), string='(', start=(1, 7), end=(1, 8), line='def foo(bar):\n')
TokenInfo(type=1 (NAME), string='bar', start=(1, 8), end=(1, 11), line='def foo(bar):\n')
TokenInfo(type=54 (OP), string=')', start=(1, 11), end=(1, 12), line='def foo(bar):\n')
TokenInfo(type=54 (OP), string=':', start=(1, 12), end=(1, 13), line='def foo(bar):\n')
TokenInfo(type=4 (NEWLINE), string='\n', start=(1, 13), end=(1, 14), line='def foo(bar):\n')
TokenInfo(type=5 (INDENT), string='    ', start=(2, 0), end=(2, 4), line='    return bar + 1\n')
TokenInfo(type=1 (NAME), string='return', start=(2, 4), end=(2, 10), line='    return bar + 1\n')
TokenInfo(type=1 (NAME), string='bar', start=(2, 11), end=(2, 14), line='    return bar + 1\n')
TokenInfo(type=54 (OP), string='+', start=(2, 15), end=(2, 16), line='    return bar + 1\n')
TokenInfo(type=2 (NUMBER), string='1', start=(2, 17), end=(2, 18), line='    return bar + 1\n')
TokenInfo(type=4 (NEWLINE), string='\n', start=(2, 18), end=(2, 19), line='    return bar + 1\n')
TokenInfo(type=6 (DEDENT), string='', start=(3, 0), end=(3, 0), line='')
TokenInfo(type=0 (ENDMARKER), string='', start=(3, 0), end=(3, 0), line='')
```

Outline

- 1 inspect
- 2 sys.settrace
- 3 sys.monitoring
- 4 tokenize
- 5 libcst**

ast vs. cst



语法分析

```
In [1]: import libcst
In [2]: libcst.parse_expression("a + 10")
Out[2]:
BinaryOperation(
  left=Name(
    value='a',
    lpar=[],
    rpar=[],
  ),
  operator=Add(
    whitespace_before=SimpleWhitespace(
      value=' ',
    ),
    whitespace_after=SimpleWhitespace(
      value=' ',
    ),
  ),
  right=Integer(
    value='10',
    lpar=[],
    rpar=[],
  ),
  lpar=[],
  rpar=[],
)

In [3]: with open("a.py") as f:
...:     code = libcst.parse_module(f.read())
```

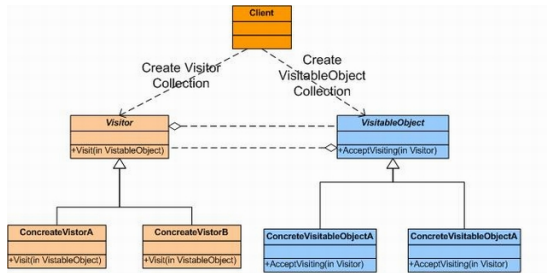
访问者模式

访问者模式是一种将算法与对象结构分离的软件设计模式¹。

这个模式的基本想法如下：首先我们拥有一个由许多对象构成的对象结构，这些对象的类都拥有一个 `accept` 方法用来接受访问者对象；访问者是一个接口，它拥有一个 `visit` 方法，这个方法对访问到的对象结构中不同类型的元素作出不同的反应；在对象结构的一次访问过程中，我们遍历整个对象结构，对每一个元素都实施 `accept` 方法，在每一个元素的 `accept` 方法中回调访问者的 `visit` 方法，从而使访问者得以处理对象结构的每一个元素。我们可以针对对象结构设计不同的实在的访问者类来完成不同的操作。

¹<https://zh.wikipedia.org/wiki/>

访问者模式



CSTVisitor

```
class FuncIntVisitor(libcst.CSTVisitor):
    def __init__(self):
        self.funcint = dict()
        self.current_func = None
    def visit_FunctionDef(self, node):
        name = node.name.value
        self.funcint[name] = 0
        self.current_func = name
    def visit_Integer(self, node):
        if self.current_func:
            self.funcint[self.current_func] += 1
    def leave_FunctionDef(self, node):
        print(self.current_func, self.funcint[self.current_func])
        self.current_func = None

text='''
def foo(bar):
    return 1
def bar():
    print('hello')

foo(3)'''

code = libcst.parse_module(text)
visitor = FuncIntVisitor()
code.visit(visitor)
print(visitor.funcint)
```


CSTTransformer

```
import libcst
import re
class FuncNameUpper(libcst.CSTTransformer):
    def leave_FunctionDef(self, node, updated):
        name = updated.name.value
        updated_name = re.sub(r'_([a-z])', lambda match: match.group(1).upper(), name)
        return libcst.FunctionDef(name=libcst.Name(value=updated_name),
                                   params=updated.params,
                                   body=updated.body)

text='''
def foo_foo_bar(bar):
    return 1
def bar():
    print('hello')

foo(3)'''

code = libcst.parse_module(text)
transformer = FuncNameUpper()
updated = code.visit(transformer)
print(updated.code)
```